

2η Εργασία Εργαστηρίου

Μικροεπεξεργαστές και Περιφερειακά

Ομάδα 40: Μιχαήλ Μπαζιάκος
AEM: 11125

19 Μαΐου 2026

1. Εισαγωγή

Στην εργασία αυτή αναπτύχθηκε firmware για ελεγκτή οπτικής σηματοδότησης. Το σύστημα δέχεται προφίλ λειτουργίας (ακολουθία ψηφίων) μέσω UART, ελέγχει LED με δυναμική συχνότητα αναβοσβήσιματος και διαθέτει διακόπτη έκτακτης ανάγκης (E-Stop) με μηχανισμό ξεκλειδώματος. Ο κώδικας υλοποιήθηκε σε C για ARM Cortex-M4 (NUCLEO) με χρήση διακοπών, ουρών και χρονομετρητή.

2. Αρχιτεκτονική Λογισμικού

Ο κώδικας οργανώνεται σε τέσσερις βασικές ενότητες: UART (λήψη χαρακτήρων μέσω διακοπής σε κυκλική ουρά), Timer (μετρητής 1 ms), GPIO (έλεγχος LED και κουμπιού) και Λογική Ελέγχου (επεξεργασία εντολών, timeout, looping, E-Stop). Ο κώδικας λειτουργεί μέσω αλλαγής και ελέγχου global μεταβλητών. Οι ISR κρατήθηκαν επίτηδες μικρές, αλλάζοντας μόνο global flags, ενώ ο έλεγχος αυτών των flags πραγματοποιείται στην `while(1)`.

3. Διαχείριση Διακοπών

Οι προτεραιότητες NVIC ρυθμίστηκαν ως: `EXTI15_10_IRQn` (κουμπί) = 0 (υψηλότερη), `SysTick_IRQn` (timer) = 1 (μεσαία), `USART2_IRQn` (UART) = 2 (χαμηλότερη). Αυτό εξασφαλίζει άμεση απόκριση στο E-Stop.

4. Λειτουργίες Συστήματος

Η λήψη εντολών γίνεται χαρακτήρα προς χαρακτήρα με αποθήκευση σε ουρά. Υπάρχει μηχανισμός timeout 4 δευτερολέπτων: αν ο χρήστης δεν πατήσει Enter, η εντολή ακυρώνεται. Η εκτέλεση προφίλ διαρκεί 2 δευτερόλεπτα ανά ψηφίο, με συχνότητα αναβοσβήσιματος 1–9 Hz (ψηφίο '0' = σβηστό). Αν η εντολή λήγει σε '-', το προφίλ επαναλαμβάνεται συνεχώς (looping) μέχρι την εκτέλεση νέας εντολής. Το κουμπί E-Stop (με rising trigger) κλειδώνει το σύστημα, ενώ το δεύτερο πάτημα επιτρέπει ξεκλείδωμα με πληκτρολόγηση "UNLOCK" εντός 5 δευτερολέπτων.

5. Προβλήματα και Λύσεις

Πρόβλημα 1 (κουπί): Αρχικά με Falling trigger και Input mode, το callback δεν εκτελούνταν διότι ο driver ελέγχει pin=1. Λύση: PullDown με Rising trigger.

Πρόβλημα 2 (αναβοσβήσιμο): Για freq=3 το duty cycle ήταν ασύμμετρο. Λύθηκε με υπολογισμό χρόνου εντός περιόδου 1 s.

Πρόβλημα 3 (λοοπινγ): Υλοποιήθηκε έλεγχος για παύλα στο τέλος και loop_enabled.

Πρόβλημα 4 (UART πριντ): Παρ' όλες τις διορθώσεις, το uart_print() εξακολουθεί να μην λειτουργεί. Ο εξεταστής ανέφερε ότι μάλλον ευθύνεται το περιβάλλον εικονικής μηχανής (VM). Οι υπόλοιπες λειτουργίες UART (λήψη, echo χαρακτήρων, αποστολή μεμονωμένων bytes) λειτουργούν κανονικά.

Πρόβλημα 5 (σβήσιμο στο 0): Υπήρχε θέμα διακοπής λειτουργίας όταν το 0 βρισκόταν ενδιάμεσα στην ουρά, διότι ο αρχικός κώδικας ανάγκαζε τη διακοπή της ουράς και το σβήσιμο του LED.

Πρόβλημα 6 (πρόβλημα στους **drivers**): Σε πολλές βιβλιοθήκες τα `#include "stm32f4xx_XXXXX.h"` ήτανε δηλωμένα με κεφαλαία ενώ έπρεπε να είναι με πεζά.

6. Δοκιμές

Ελέγχθηκε αποστολή "52" (σωστή εκτέλεση), "41" με αναμονή 4 s (timeout), "415-" (looping), "101" για αγνόηση του 0, πάτημα E-Stop (lock και LED σταθερό), δεύτερο πάτημα με "UNLOCK" (ξεκλείδωμα). Για ψηφία 1-9 το LED αναβόσβησε με σωστή συχνότητα, για '0' έμεινε σβηστό.

7. Συμπεράσματα

Ο κώδικας πληροί τις προδιαγραφές πλην του uart_print() που παραμένει ανενεργό. Οι υπόλοιπες λειτουργίες (E-Stop, looping, timeout, εκτέλεση προφίλ) υλοποιήθηκαν επιτυχώς. Το πρόβλημα UART αποδίδεται πιθανότατα στο Virtual COM port και θα επιλυθεί με περαιτέρω διερεύνηση.

8. Παραδοτέα

Συμπεριλαμβάνονται στο συμπιεσμένο αρχείο:

- Το αρχείο main.c με τον πλήρη κώδικα.
- Η παρούσα αναφορά σε μορφή PDF.